Data transformation and dimensionality reduction (DTDR) II

Introduction

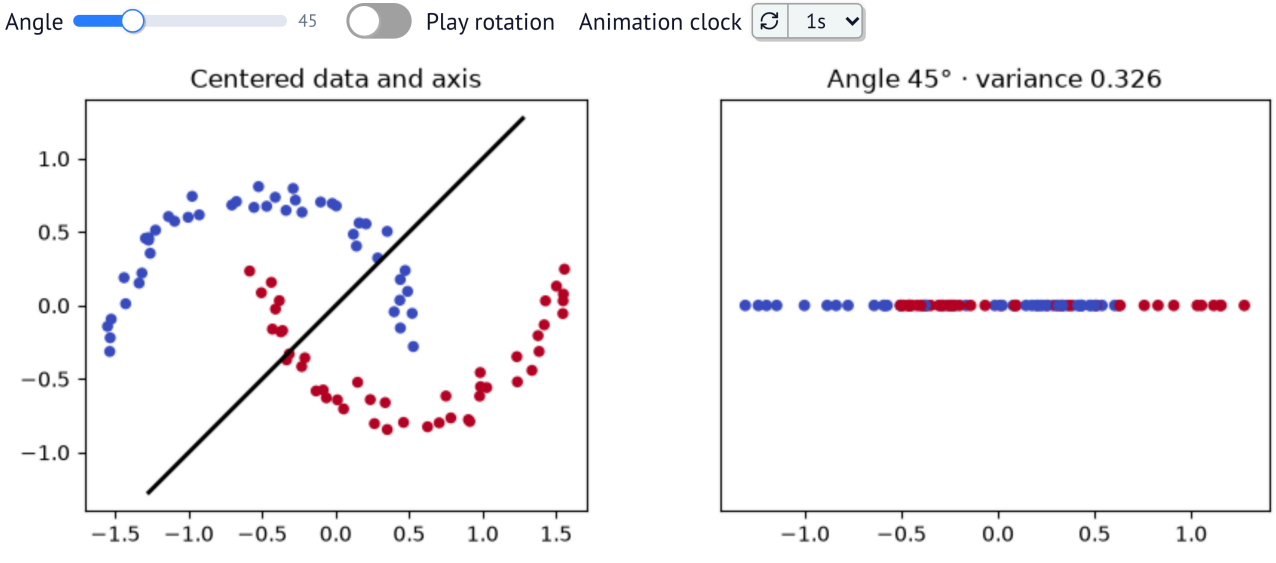
- Last lecture we looked at compression of data.
    - Reduce the dimensionality.
- Learned about two methods:
    - Fisher Discriminant Analysis (need labeled data).
    - Principal Component Analysis (PCA).
- Both methods use a linear transformation.
- Today, we will learn about non-linear DTDR techniques:
    - Kernel PCA (1998)
    - Laplacian Eigenmaps (2003)

Recall PCA

- Dataset with N observations $\mathbf{x}_1, \ldots, \mathbf{x}_N$ with $\mathbf{x}_i \in \mathbb{R}^d$
- Aim: reduce dimensionality!
- How: project $\mathbf{x}_i$ into the direction $\mathbf{v}$ along with the variance of the data is maximum $\rightarrow z_i = \mathbf{v}^T \mathbf{x}_i$
- Direction that maximise the variance: $\Sigma \mathbf{v} = \lambda \mathbf{v}$, with $\mathbf{\Sigma}$ covariance matrix $\rightarrow$ **first eigenvector of the covariance matrix**.

PCA projections are linear wrt the original datapoints; this can be not enough when data represents non linear structures.

$$\mathbf{e}(\theta) = (\cos\theta, \sin\theta)^T, \qquad z_i = \mathbf{e}(\theta)^T \widetilde{\mathbf{x}}_i.$$



Kernel PCA

- Non-linear operations with respect to $\mathbf{x}$.
- Let $\phi : \mathbb{R}^d \rightarrow \mathbb{H}$ be a data transformation function from $\mathbb{R}^d$ to the feature space $\mathbb{H}$
- KPCA $\rightarrow$ apply PCA to transformed points $\phi(\mathbf{x}_1), \ldots, \phi(\mathbf{x}_N)$

Eigenvectors of autocorrelation matrix R

- Let $\mathbf{R} = \frac{1}{N} \sum_{i=1}^{N} \phi(\mathbf{x}_i)\phi(\mathbf{x}_i)^T$ be the autocorrelation matrix in the feature space $\mathbb{H}$;
- Eigendecomposition: $\mathbf{R}\mathbf{v} = \lambda\mathbf{v}$
- $\mathbf{R}$ can have very high dimension $\rightarrow$ construct it and diagonalize it can be computationally infeasible!
- **Trick**: 1) we can show that the eigenvectors associated to non-null eigenvalues of $\mathbf{R}$ are linear combination of the features; 2) the coefficient of this linear combination can be obtained from the kernel matrix $\mathbf{K}$.

1) Eigenvectors of $\mathbf{R}$ are linear combination of the features.

- $\mathbf{v}$ $=$ $\sum_{i=1}^{N} a_i \phi$ $(\mathbf{x}_i$ $)$ with a_i $= \frac{1}{N\lambda} \phi$ $(\mathbf{x}_i$ $)^T \mathbf{v}$

2) How to obtain the coefficient of this linear combination

- Introduce the kernel function $k(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$ and the associated kernel matrix $\mathbf{K}$.
- $\mathbf{K}\mathbf{a} = N\lambda\mathbf{a}$

KPCA

$$\mathbf{R}\mathbf{v} = \lambda\mathbf{v}$$

$$\mathbf{v} = \sum_i a_i \phi(\mathbf{x}_i)$$

For finding the coefficients:

$$\mathbf{K}\mathbf{a} = \tilde{\lambda}\mathbf{a}$$

with:

$$\tilde{\lambda} = N\lambda$$

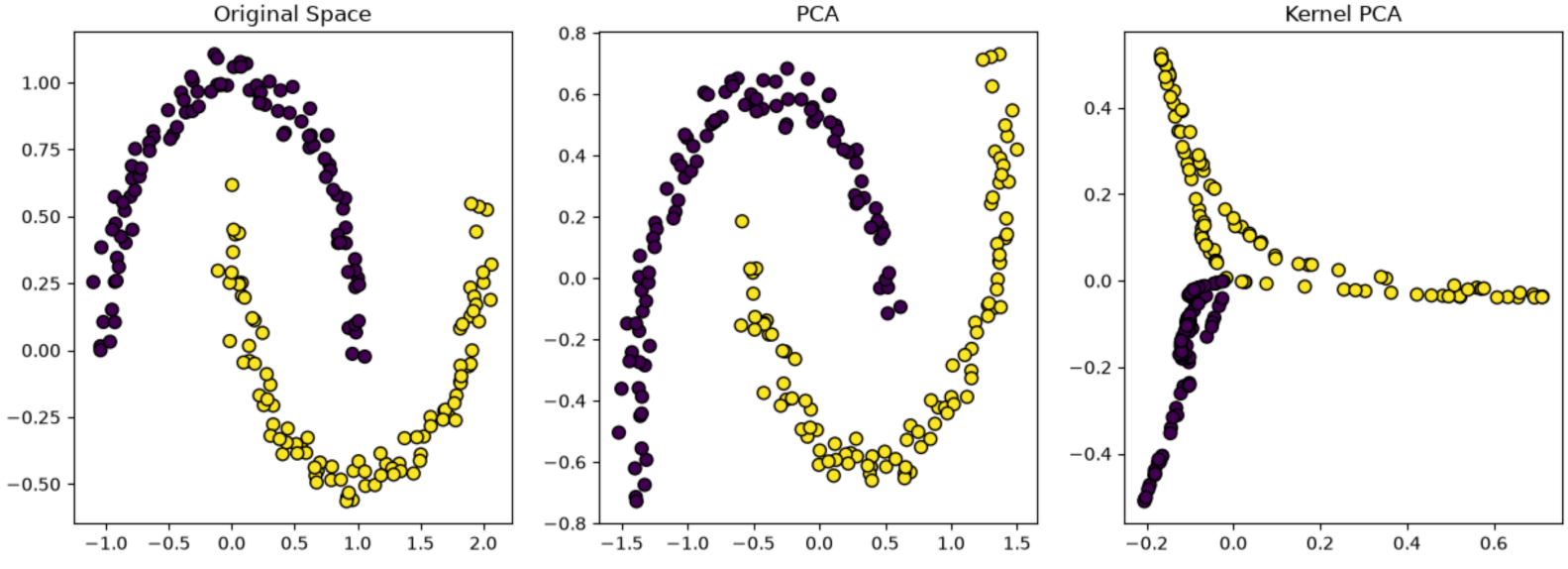
Finally, we project:

$$z(\mathbf{x}) = \mathbf{v}^T \phi(\mathbf{x})$$

Substituting:

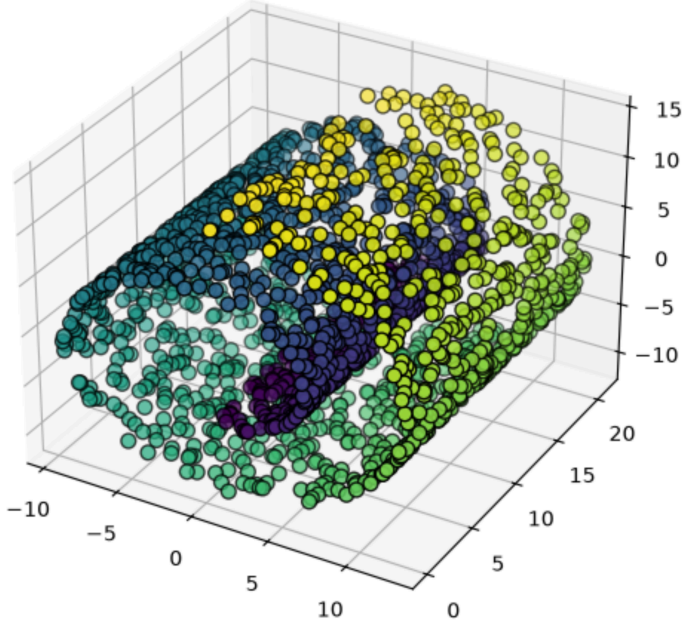
$$\boxed{z(\mathbf{x}) = \sum_i a_i k(\mathbf{x}_i, \mathbf{x})}$$

Visualizing in $\mathbb{H}$


Original Space
PCA
Kernel PCA
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−1.0
−0.5
0.0
0.5
1.0
1.5
2.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6
−0.8
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5
0.4
0.2
0.0
−0.2
−0.4
−0.2
0.0
0.2
0.4
0.6

Laplacian Eigenmaps

- Transform $\mathbf{x} \in \mathbb{R}^d$ into $\mathbf{z} \in \mathbb{R}^k$, where $k \leq d$
- Such that: neighbors in $\mathbb{R}^d$ stay neighbors in $\mathbb{R}^k$



Static notebook

This notebook is not connected t
interactive elements will not wor

Adjacency matrix

- What makes points neighbors?
    - Similarity!
- Construct graph $\mathcal{G}$ where each node i represents a datapoint $\mathbf{x}_i$; an edge connects two nodes if the two correspondent points are close (e.g $||\mathbf{x}_i - \mathbf{x}_j|| < \epsilon$)
- The correspondent adjacency matrix is a weighted symmetric matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$ with

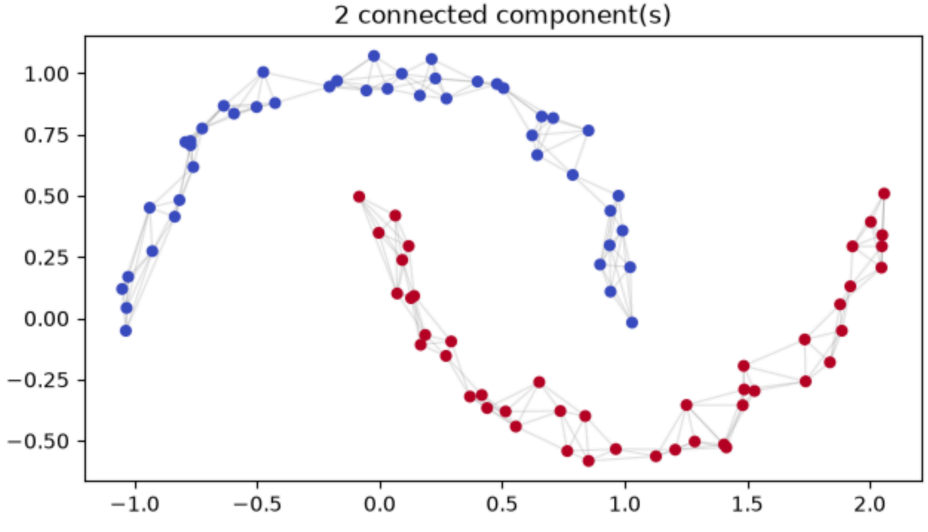
$$W_{ij} = W_{ji} \geq 0$$

Change the number of nearest neighbors.

- Too few edges: disconnected graph.
- Too many edges: unwanted shortcuts.

The number of zero Laplacian eigenvalues equals the number of components.

Nearest neighbors 5



Mapping $\mathbf{x}_i$ to $\mathbf{z}_i$

- Need a way to optimize the position of points in $\mathbb{R}^k$.
- Consider the following: $J(\mathbf{z}) = \frac{1}{2}\sum_{i=1}^{N}\sum_{j=1}^{N} W_{ij}||\mathbf{z}_i - \mathbf{z}_j||^2$

Optimizing the position of points in $\mathbb{R}^k$

- Reasonable to minimize $J(\mathbf{z})$ with respect to $\mathbf{z}$
- Obvious solution not interesting.
    - What is the obvious solution?

Optimizing the position of points in $\mathbb{R}^k$

$$\min_{z \in \mathbb{R}^N} \mathbf{z}^T \mathbf{L} \mathbf{z}$$

with

$$\mathbf{z}^T \mathbf{D} \mathbf{z} = 1$$

and

$$\mathbf{z}^T \mathbf{D} \mathbf{1} = 0$$

The Laplacian matrix $\mathbf{L}$

- Deep theory!
- The Laplacian tells us the divergence of the gradients.
    - How much is the steepness changing.
- Analogous to the second derivative for a continuous, single variate function.
- **Nice resource for further information**

Lagrange function for Laplacian Eigenmaps

Lagrange function:

$$L(\mathbf{z}, \lambda) = \mathbf{z}^T \mathbf{L}\mathbf{z} - \lambda(\mathbf{z}^T \mathbf{D}\mathbf{z} - 1)$$

Set derivative to zero:

$$\frac{\partial L}{\partial \mathbf{z}} = 2\mathbf{L}\mathbf{z} - 2\lambda\mathbf{D}\mathbf{z} = 0$$

Which gives:

$$\mathbf{L}\mathbf{z} = \lambda\mathbf{D}\mathbf{z}$$

- The minimization of this objective is given by eigenvector corresponding to the second smallest eigenvalue
- Assuming a connected graph.